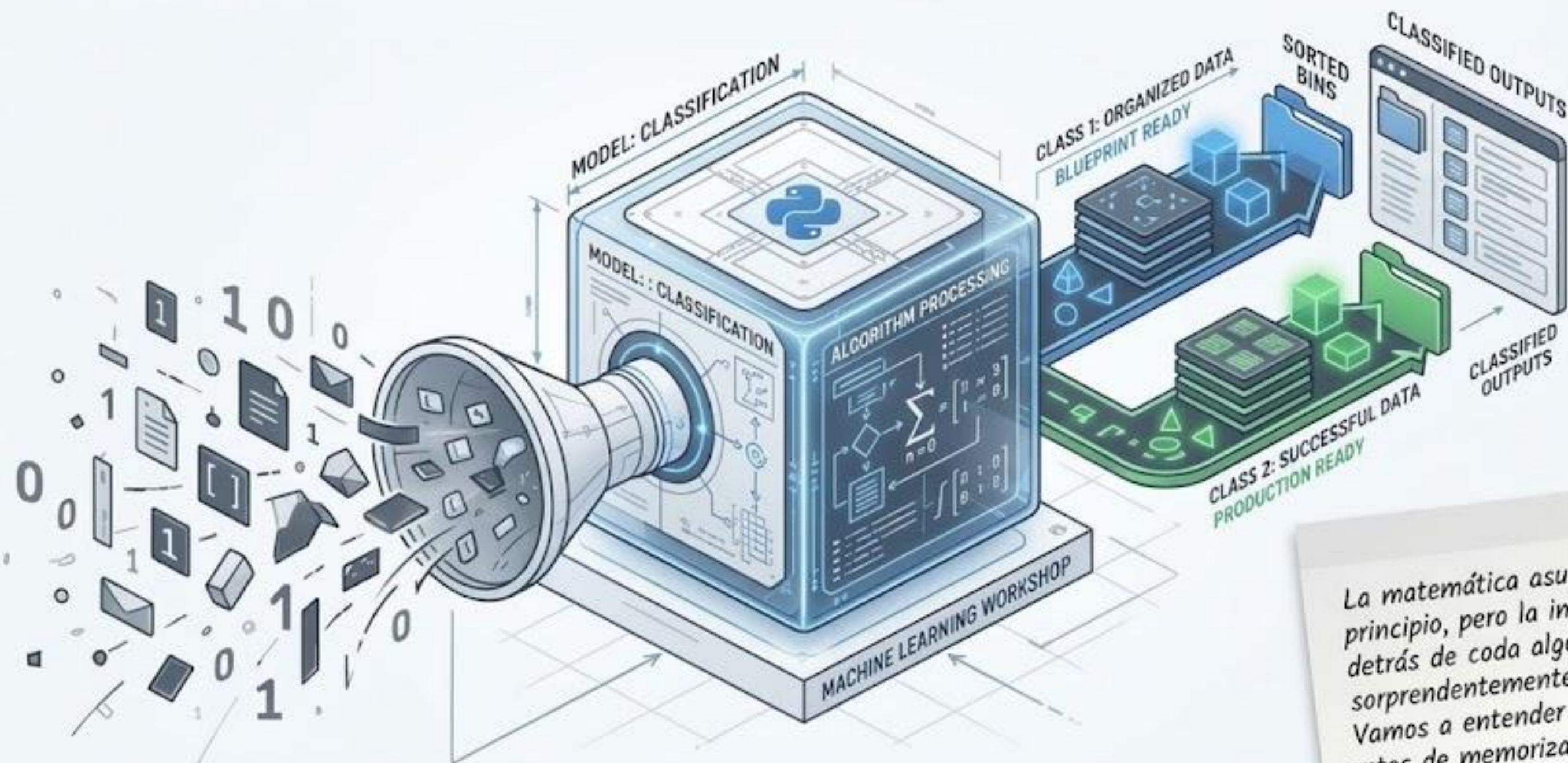


Guía de Supervivencia en Machine Learning

Clasificación Supervisada con Scikit-Learn. Desde la intuición matemática hasta el código en producción.

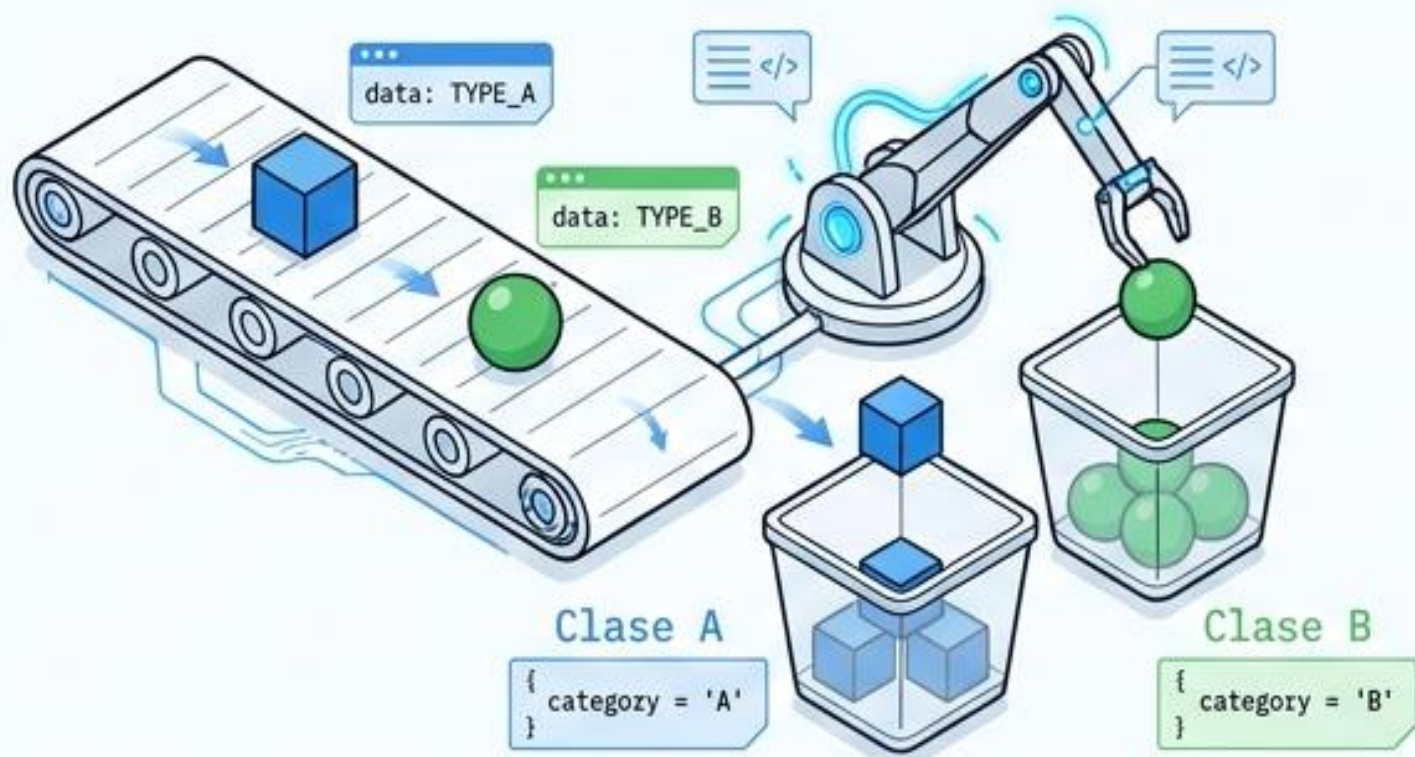


La matemática asusta al principio, pero la intuición detrás de cada algoritmo es sorprendentemente simple. Vamos a entender el porqué antes de memorizar el código.

¿Qué significa realmente Clasificar?

Clasificación

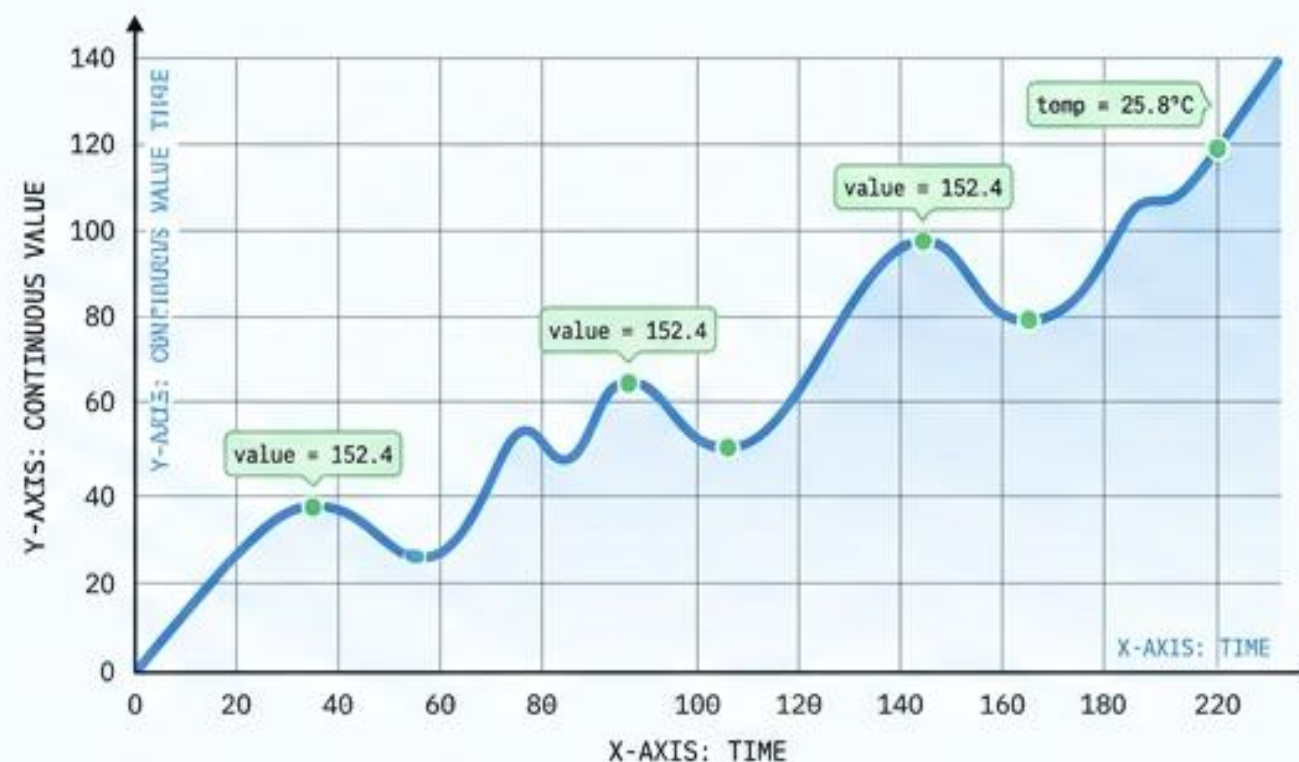
Predice etiquetas discretas



- > - ¿Gripe o COVID?
- > - ¿Aprobar o Rechazar?
- > - ¿Bandeja de entrada o Spam?

Regresión

Predice valores numéricos continuos



- > - ¿Cuál será el precio de esta casa?
- > - ¿Qué temperatura hará mañana?

En Clasificación Supervisada, le damos a la máquina datos de entrada (síntomas) y la respuesta correcta (diagnóstico) para que aprenda a asignar categorías.

La Anatomía del Aprendizaje



Features (X) - Las características

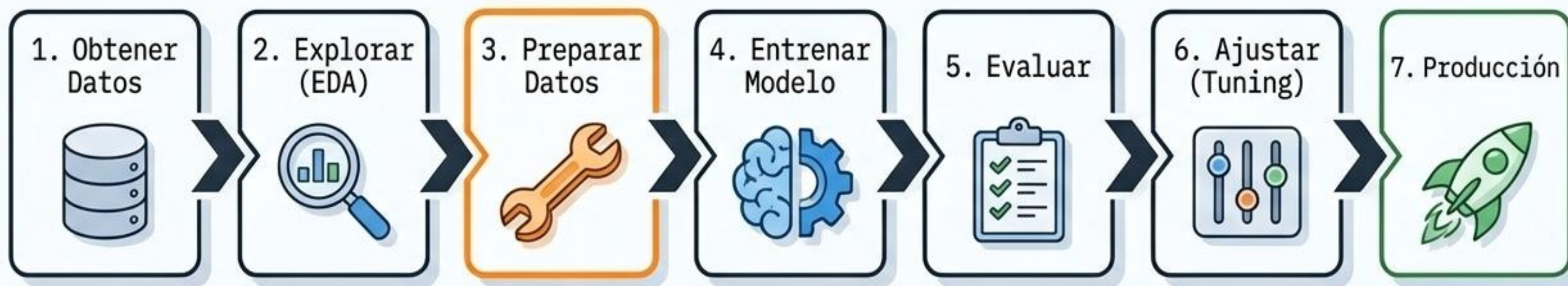
Target (y) - La etiqueta a predecir

¿Qué es entrenar? Es mostrarle a la máquina miles de flashcards con sus respuestas correctas para que descubra los patrones internos.

Features (X) - Las características				Target (y) - La etiqueta a predecir
largo_sépalo	ancho_sépalo	largo_pétalo	ancho_pétalo	species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
7.0	3.2	4.7	1.4	versicolor
6.4	3.2	4.5	1.5	versicolor
5.8	2.7	5.1	1.9	virginica

Una Observación
(un ejemplo o registro)

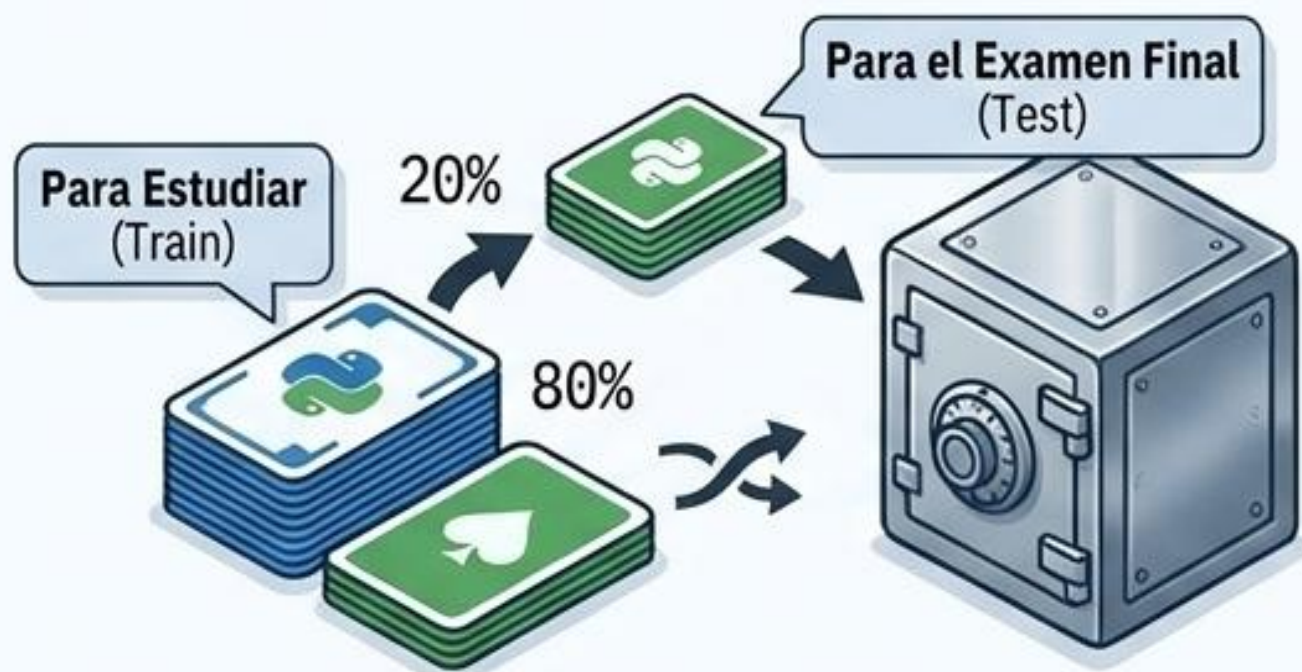
El Flujo de Trabajo (End-to-End)



¡Atención al Paso 3!

Cada paso es crucial. Un error en la preparación de datos se arrastrará inevitablemente como un virus hasta arruinar la evaluación final en el paso 5.

Las Dos Reglas de Oro



1. Train/Test Split

Nunca evalúes con los datos que usaste para entrenar; el modelo solo los memorizaría.

```
X_train, X_test = train_test_split(X, y,  
                                  test_size=0.2, random_state=42)
```

¿Por qué 42? Es
convención para
asegurar
reproducibilidad.

2. Estandarización (StandardScaler)



fit_transform SOLO en **Train**.
transform en **Test**. Hacer fit en
Test causa Fuga de Información
(Data Leakage) desde el futuro.

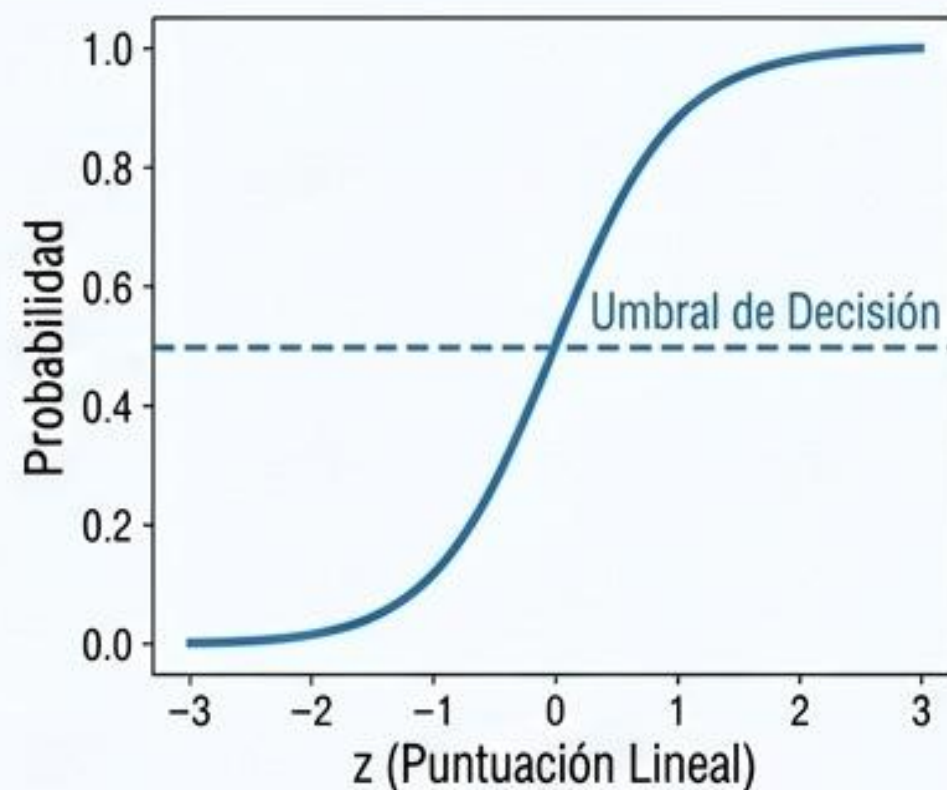
1. Regresión Logística (El Baseline Probabilístico)

La Intuición



Calcula una **puntuación lineal** basada en las features y luego la aplasta para convertirla matemáticamente en una probabilidad exacta entre 0 y 1.

El Motor Matemático



$$P = \frac{1}{1 + e^{-z}}$$

Si $P \geq 0.5 \rightarrow$ Clase 1.
Si $P < 0.5 \rightarrow$ Clase 0.

Ficha Técnica

```
modelo = LogisticRegression  
(max_iter=1000)  
  
modelo.fit(X_train_scaled,  
y_train)
```

Pros:

Simple, rápido, da probabilidades reales.

Contras:

Sensible a outliers, requiere datos linealmente separables.

2. Árboles de Decisión (El Juego de las 20 Preguntas)

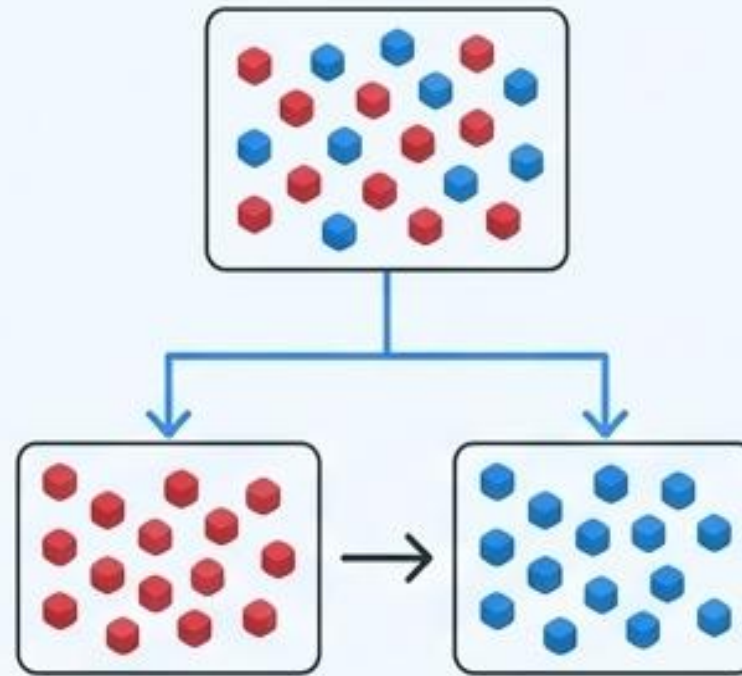
La Intuición



Es el algoritmo más interpretable humano-máquina.

Divide los datos haciendo preguntas secuenciales.

El Motor Matemático



Busca la pregunta que maximice la Ganancia de Información.

Utiliza métricas como la Entropía (desorden) o el Índice Gini (impureza) para encontrar el corte perfecto en los datos.

Ficha Técnica

```
arbol =  
DecisionTreeClassifier(max_dep  
th=5)
```

Pros:

No necesita escalar los datos. Altísima interpretabilidad.

Contras:

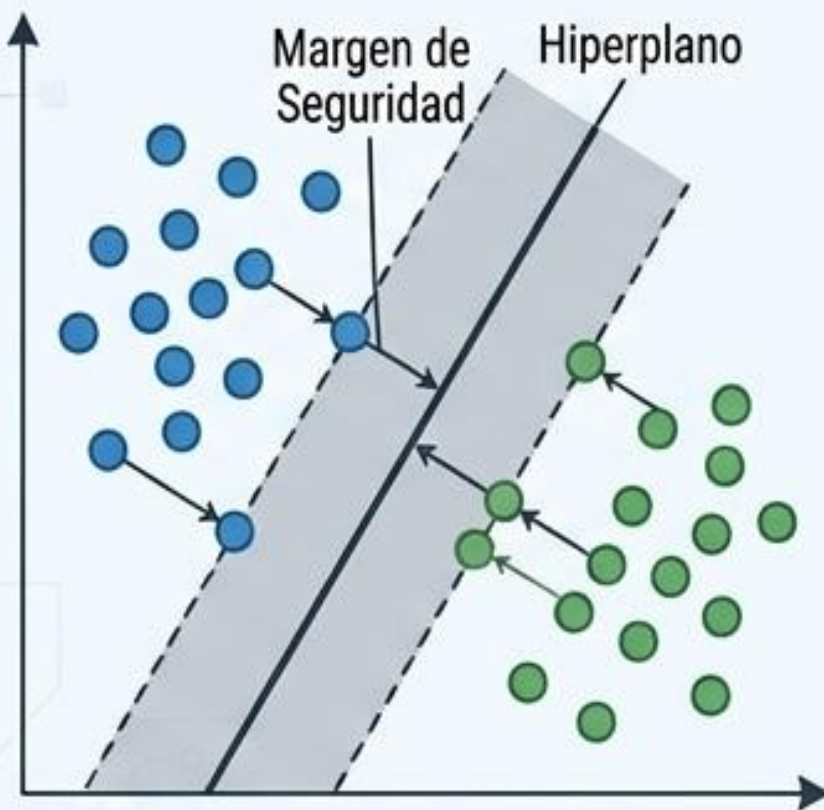
Propenso al Overfitting masivo si no se controla el parámetro max_depth.



Un árbol sin límites memoriza cada dato y fracasa en producción.

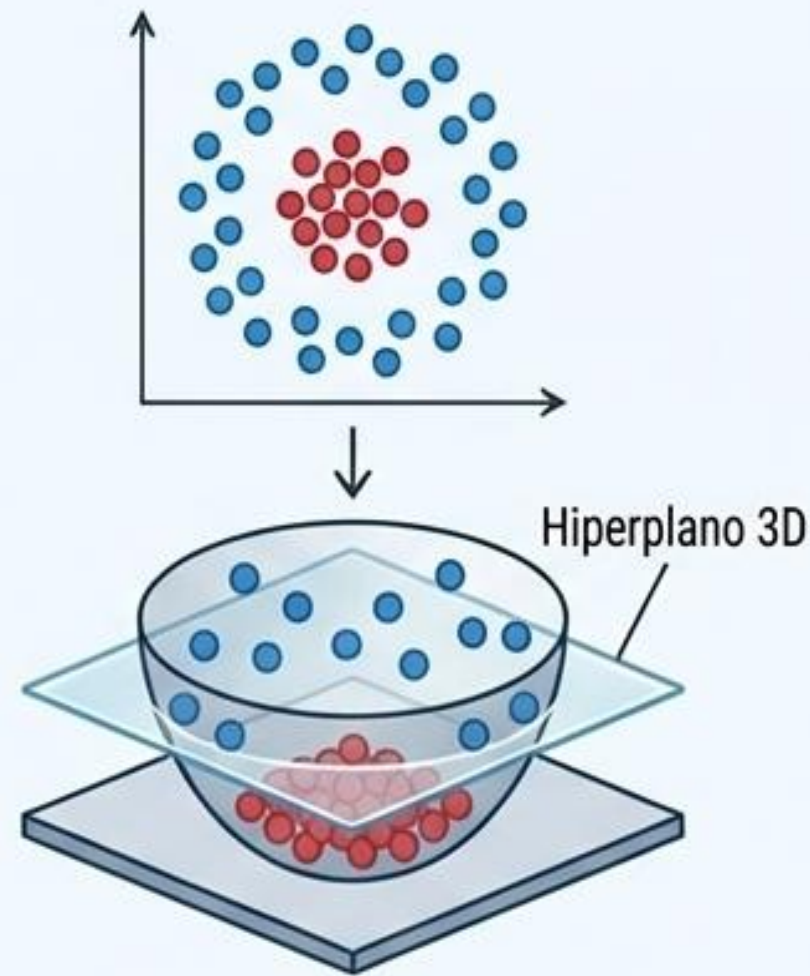
3. Support Vector Machines (La Frontera Máxima)

La Intuición



Busca dibujar la frontera (hiperplano) que maximice el **margen de seguridad** entre las diferentes clases.

El Truco del Kernel



¿Datos no lineales? El Kernel RBF proyecta los datos a dimensiones superiores donde sí son linealmente separables.

Ficha Técnica

```
svm = SVC(kernel='rbf',  
C=1.0)
```

Pros:

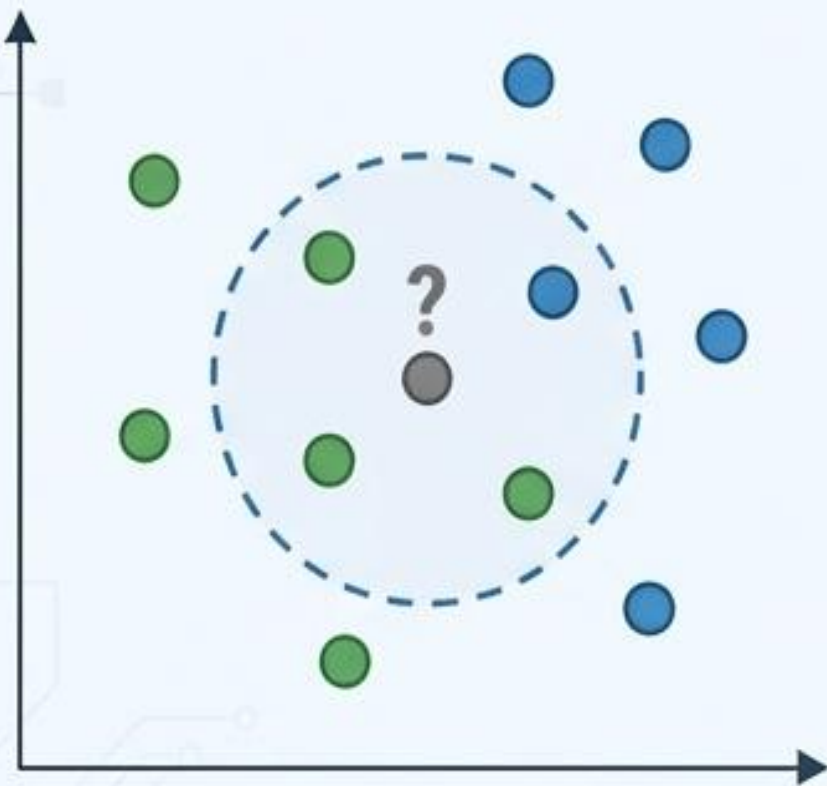
Extremadamente potente en espacios complejos y alta dimensionalidad.

Contras:

Caja negra (baja interpretabilidad), requiere mucha memoria, **REQUIERE estandarización.**

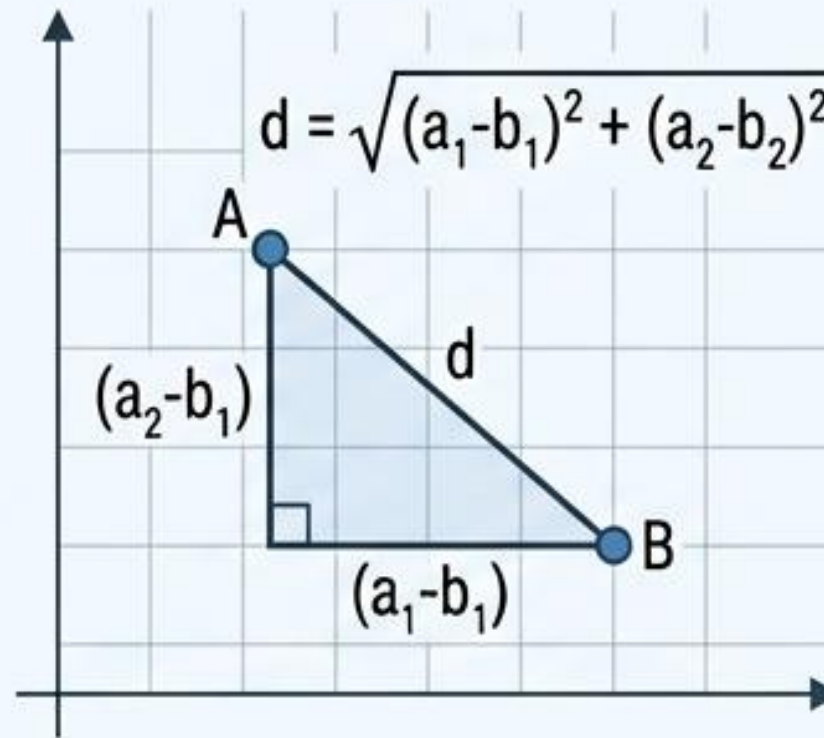
4. K-Nearest Neighbors (Democracia Local)

La Intuición



Es un algoritmo perezoso. Para clasificar un punto nuevo, simplemente le pregunta a sus K vecinos más cercanos y asigna la clase elegida por mayoría de votos.

El Motor Matemático



! ¡Escalar es OBLIGATORIO!

Dado que KNN mide distancias geométricas, si una feature mide milímetros y otra kilómetros, la distancia estará dominada por los kilómetros y el modelo ignorará los milímetros.

Ficha Técnica

```
knn = KNeighborsClassifier  
(n_neighbors=5)
```

Pros:

Sin fase de entrenamiento real, captura geometrías complejas y no lineales sin esfuerzo.

Contras:

Extremadamente lento en predicción con datasets masivos.

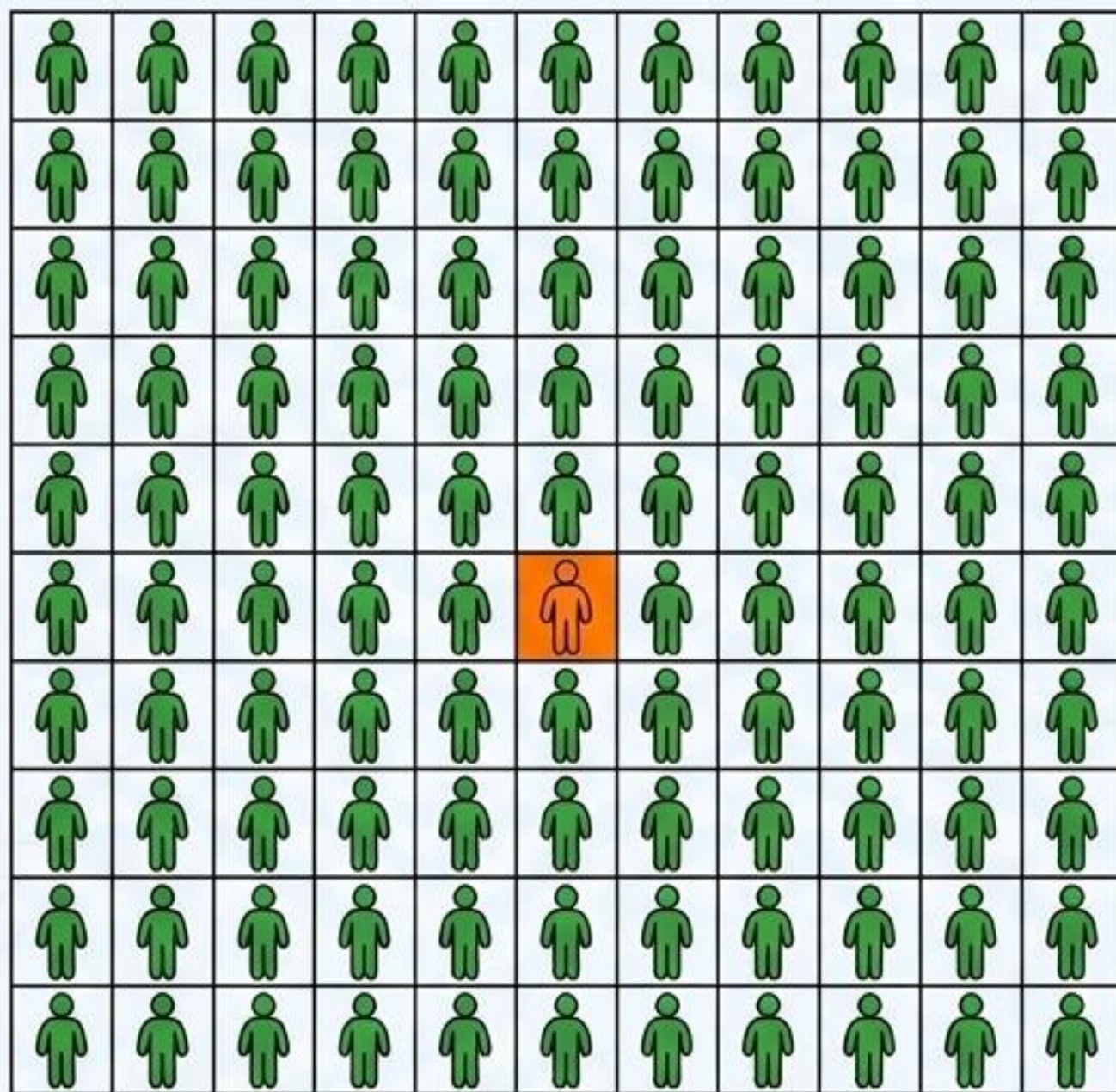
Síntesis: ¿Qué Herramienta Elijo?

Criterio	Regresión Logística	Árbol de Decisión	SVM (Kernel RBF)	KNN
Interpretabilidad	★ ★ ★	★ ★ ★	★	★ ★
Relaciones No Lineales	★	★ ★ ★	★ ★ ★	★ ★ ★
Eficiencia en Datos Masivos	★ ★ ★	★ ★ ★	★ (Lento)	★ (Lento)
¿Necesita Escalar Datos?	SÍ	NO	SÍ	SÍ

Recomendación de Ingeniería:

Empieza siempre con Regresión Logística como tu base (baseline). Luego, utiliza **Árboles** si el cliente necesita entender exactamente por qué se tomó una decisión, o escala a **SVM/KNN** si la geometría del problema es altamente compleja y priorizas la precisión sobre la interpretabilidad.

La Trampa del Accuracy (Exactitud)



¿Genio o Inútil?

Imagina un modelo 'tonto' que está programado para decir SIEMPRE 'Sano', sin importar los datos que reciba.

Aciertos: 99 / 100

Accuracy Matemático: 99%

Desempeño Real en Fraude/Cáncer:
0% de detección.

El **Accuracy es ciego al contexto** cuando las clases están desbalanceadas. Ignorar el 1% rojo es catastrófico, aunque tengas una nota casi perfecta. **Necesitamos métricas que diagnostiquen la calidad de nuestros errores.**

Dashboard de Diagnóstico: La Matriz de Confusión

Matriz de Confusión

Realidad	Verdaderos Positivos (TP) Dijo Cáncer, Era Cáncer ✓	Falsos Negativos (FN) Dijo Sano, Era Cáncer (Peligro Fatal)
	Falsos Positivos (FP) Dijo Cáncer, Era Sano (Susto/Falsa Alarma)	Verdaderos Negativos (TN) Dijo Sano, Era Sano ✓
	Predicción	

Métricas Derivadas

 **Precision** = $TP / (TP + FP)$

Priorízala si las Falsas Alarmas son costosas (ej. Filtro Spam).

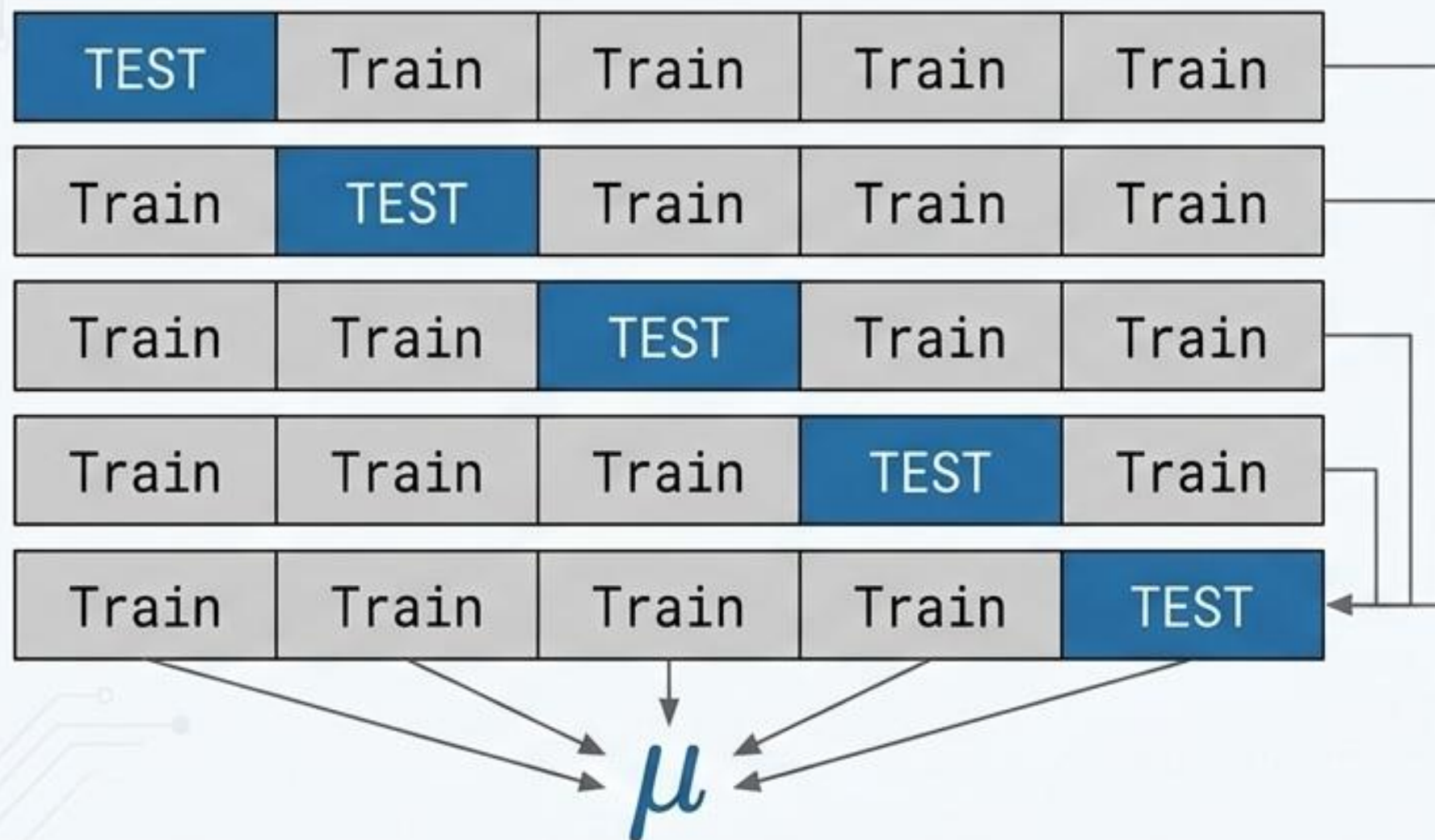
 **Recall** = $TP / (TP + FN)$

Priorízala si los Descuidos son peligrosos (ej. Diagnóstico Médico).

 **F1-Score** = Media armónica

Úsala para buscar el equilibrio perfecto entre Precision y Recall.

Subiendo de Nivel: Validación Cruzada (K-Fold CV)



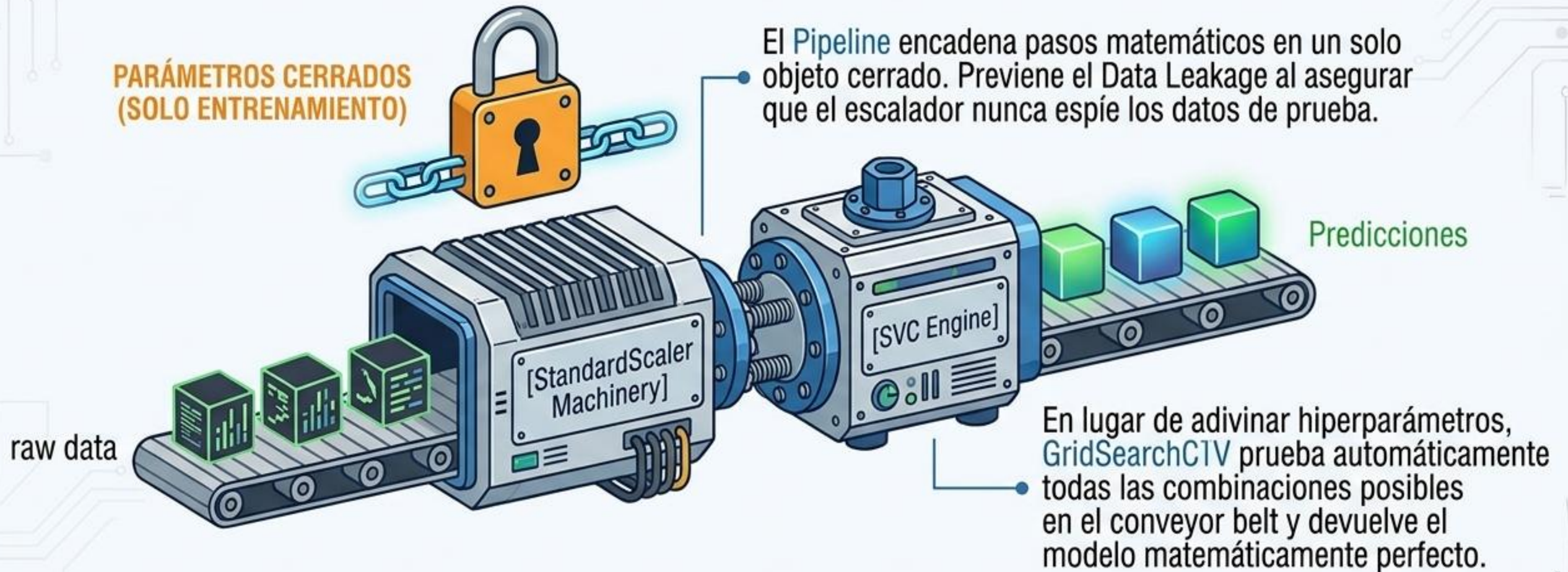
Un solo corte de Train/Test puede tener “suerte” dejando los datos fáciles en la validación.

K-Fold rota sistemáticamente el bloque de prueba K veces para demostrar que el modelo es consistentemente robusto.

```
scores = cross_val_score(modelo, X, y, cv=5)  
> F1 promedio: 0.92 ± 0.03
```

El $\pm$ (desviación estándar) demuestra la estabilidad del modelo.

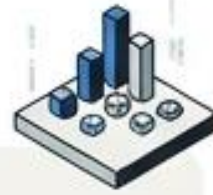
Ingeniería de Producción: Pipelines y GridSearchCV



```
param_grid = {'classifier__C': [0.1, 1, 10], 'classifier__kernel': ['linear', 'rbf']}  
grid = GridSearchCV(pipeline, param_grid, cv=5)
```

Tu Zona de Prácticas (Datasets en Scikit-Learn)

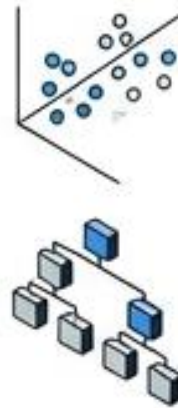
Nivel Básico: Iris



```
load_iris()
```

150 muestras | 3 Clases

El 'Hola Mundo'.
Perfecto para entender
la mecánica básica y
visualizar Árboles de
Decisión.



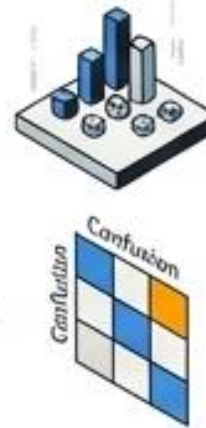
Nivel Intermedio: Breast Cancer



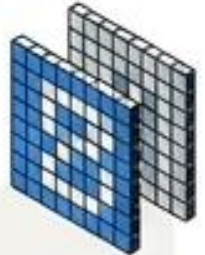
```
load_breast_cancer()
```

569 muestras | 2 Clases
(Desbalanceado)

El simulador médico.
Crucial para dominar
Recall, Matrices de
Confusión y el F1-Score.



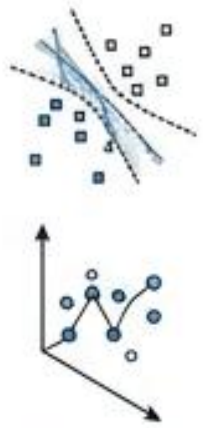
Nivel Avanzado: Digits



```
load_digits()
```

1,797 muestras | 10 Clases
(Imágenes 8x8)

El desafío de píxeles.
Ideal para experimentar
el poder puro de SVM y
KNN en alta
dimensionalidad.



Machine Learning no es magia, es ingeniería aplicada. Has aprendido los cimientos matemáticos, las herramientas y las trampas. Ahora, importa Scikit-Learn y empieza a construir. ¡Éxitos en Ingeniería del Conocimiento!